

UNIT III: Synchronization Tools & Deadlocks

Synchronization Tools: The Critical Section Problem, Peterson's Solution, Mutex Locks, Semaphores, Monitors, Classic problems of Synchronization.

Deadlocks: System Model, Deadlock characterization, Methods for handling Deadlocks, Deadlock prevention, Deadlock avoidance, Deadlock detection, Recovery from Deadlock.

Synchronization Tools

Generally process is categorized into two types on the basis of synchronization and these are given below:

- Independent Process
- Cooperative Process

Independent Processes :- Two processes are said to be independent if the execution of one process does not affect the execution of another process.

Cooperative Processes :- Two processes are said to be cooperative if the execution of one process affects the execution of another process. These processes need to be synchronized so that the order of execution can be guaranteed.

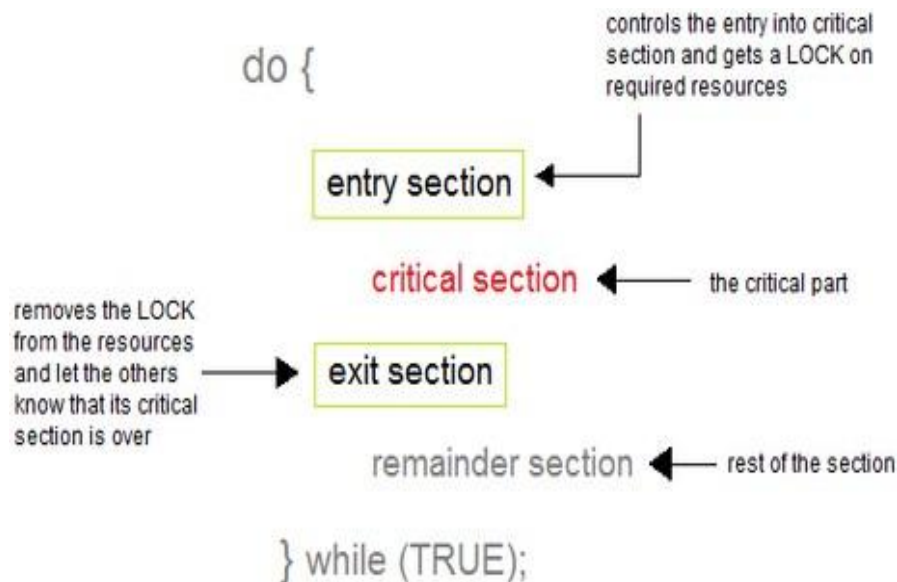
Process synchronization is the task phenomenon of coordinating the execution of processes in such a way that no two processes can have access to the same shared data and resources.

- It is a procedure that is involved in order to preserve the appropriate order of execution of cooperative processes.
- In order to synchronize the processes, there are various synchronization mechanisms.
- Process Synchronization is mainly needed in a multi-process system when multiple processes are running together, and more than one process try to gain access to the same shared resource or any data at the same time.

Race Condition :- At the time when more than one process is either executing the same code or accessing the same memory or any shared variable; In that condition, there is a possibility that the output or the value of the shared variable is wrong so for that purpose all the processes are doing the race to say that my output is correct. This condition is commonly known as a race condition.

Critical Section Problem

A Critical Section is a code segment that accesses shared variables and has to be executed as an atomic action. It means that in a group of cooperating processes, at a given point of time, only one process must be executing its critical section. If any other process also wants to execute its critical section, it must wait until the first one finishes. The entry to the critical section is mainly handled by `wait()` function while the exit from the critical section is controlled by the `signal()` function.



Entry Section :- In this section mainly the process requests for its entry in the critical section.

Exit Section :- This section is followed by the critical section.

The solution to the Critical Section Problem :-

A solution to the critical section problem must satisfy the following three conditions:

- **Mutual Exclusion** :- Out of a group of cooperating processes, only one process can be in its critical section at a given point of time.
- **Progress** :- If no process is in its critical section, and if one or more threads want to execute their critical section then any one of these threads must be allowed to get into its critical section.
- **Bounded Waiting** :- After a process makes a request for getting into its critical section, there is a limit for how many other processes can get into their critical section, before this process's request is granted. So after the limit is reached, the system must grant the process permission to get into its critical section.

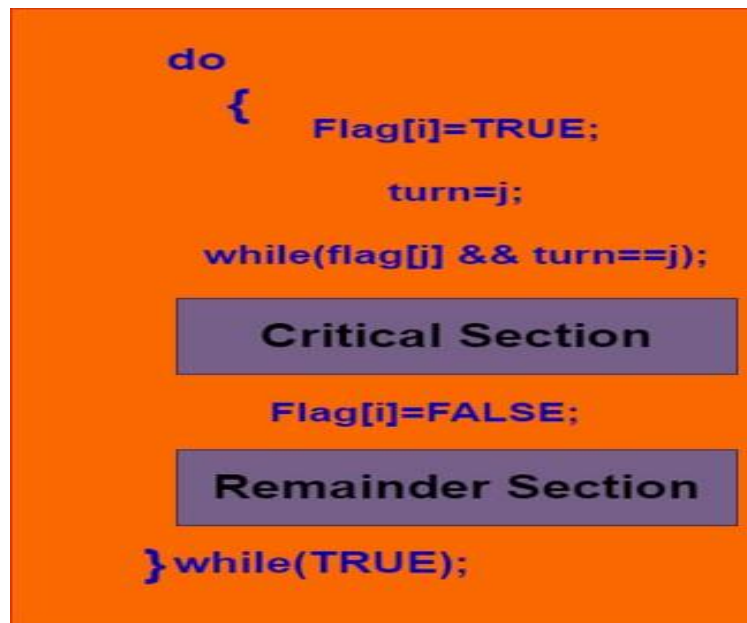
Peterson's Solution

This is widely used and software-based solution to critical section problems. Peterson's solution was developed by a computer scientist Peterson that's why it is named so.

With the help of this solution whenever a process is executing in any critical state, then the other process only executes the rest of the code, and vice-versa can happen. This method also helps to make sure of the thing that only a single process can run in the critical section at a specific time.

This solution preserves all three conditions:

- Mutual Exclusion is comforted as at any time only one process can access the critical section.
- Progress is also comforted, as a process that is outside the critical section is unable to block other processes from entering into the critical section.
- Bounded Waiting is assured as every process gets a fair chance to enter the Critical section.



The above shows the structure of process P_i in Peterson's solution.

- Suppose there are N processes ($P_1, P_2, \dots, P_N$) and as at some point of time every process requires to enter in the **Critical Section**
- A **FLAG[]** array of size N is maintained here which is by default false. Whenever a process requires to enter in the critical section, it has to set its flag as true. Example: If P_i wants to enter it will set **FLAG[i]=TRUE**.
- Another variable is called **TURN** and is used to indicate the process number that is currently waiting to enter into the critical section.
- The process that enters into the critical section while exiting would change the **TURN** to another number from the list of processes that are ready.
- Example: If the turn is 3 then P_3 enters the Critical section and while exiting $\text{turn}=4$ and therefore P_4 breaks out of the wait loop.

Mutex Locks

Most systems offer a software API equivalent called mutex locks or simply mutexes (mutual exclusion) for solving critical section problem. The terminology when using mutexes is to acquire a lock prior to entering a critical section, and to release it when exiting, the following algorithm shows Solution to the critical-section problem using mutex locks.

```

do {
    acquire lock
    critical section
    release lock
    remainder section
} while (TRUE);
  
```

Solution to the critical-section problem using mutex locks

the acquire step will block the process if the lock is in use by another process, and both acquire and release operations are atomic. Acquire and release can be implemented as shown here, based on a boolean variable "available"

Acquire:

```

acquire() {
    while (!available)
        ; /* busy wait*/
    available = false;;
}

```

Release:

```

release() {
    available = true;
}

```

One problem with the implementation shown here, is the busy loop used to block processes in the acquire phase. These types of locks are referred to as spinlocks, because the CPU just sits and spins while blocking the process. Spinlocks are wasteful of CPU cycles, and are a really bad idea on single cpu single-threaded machines, because the spinlock blocks the entire computer, and doesn't allow any other process to release the lock. On the other hand, spinlocks do not incur the overhead of a context switch, so they are effectively used on multi-threaded machines when it is expected that the lock will be released after a short time.

Semaphores

Semaphores are integer variables that are used to solve the critical section problem by using two atomic operations, wait and signal that are used for process synchronization.

The definitions of wait and signal are as follows:

Wait :-The wait operation decrements the value of its argument S, if it is positive. If S is negative or zero, then no operation is performed.

wait(S)

```

{
    while (S<=0);

    S--;
}

```

Signal :-The signal operation increments the value of its argument S.

signal(S)

```

{
    S++;
}

```

Types of Semaphores :-

There are two main types of semaphores : counting semaphores and binary semaphores.

Counting Semaphores :- These are integer value semaphores and have an unrestricted value domain. These semaphores are used to coordinate the resource access, where the semaphore count is the number of available resources. If the resources are added, semaphore count automatically incremented and if the resources are removed, the count is decremented.

Binary Semaphores :- The binary semaphores are like counting semaphores but their value is restricted to 0 and 1. The wait operation only works when the semaphore is 1 and the signal operation succeeds when semaphore is 0. It is sometimes easier to implement binary semaphores than counting semaphores.

Advantages of Semaphores :- Some of the advantages of semaphores are as follows:

- Semaphores allow only one process into the critical section. They follow the mutual exclusion principle strictly and are much more efficient than some other methods of synchronization.
- There is no resource wastage because of busy waiting in semaphores as processor time is not wasted unnecessarily to check if a condition is fulfilled to allow a process to access the critical section.
- Semaphores are implemented in the machine independent code of the microkernel. So they are machine independent.

Disadvantages of Semaphores :- Some of the disadvantages of semaphores are as follows :

- Semaphores are complicated so the wait and signal operations must be implemented in the correct order to prevent deadlocks.
- Semaphores are impractical for last scale use as their use leads to loss of modularity. This happens because the wait and signal operations prevent the creation of a structured layout for the system.
- Semaphores may lead to a priority inversion where low priority processes may access the critical section first and high priority processes later.

Monitors

Monitors are a synchronization construct that were created to overcome the problems caused by semaphores such as timing errors. Monitors are abstract data types and contain shared data variables and procedures. The shared data variables cannot be directly accessed by a process and procedures are required to allow a single process to access the shared data variables at a time. This is demonstrated as follows:

monitor monitorName

{

data variables;

Procedure P1(....)

{

}

Procedure P2(....)

{

}

Procedure Pn(....)

{

}

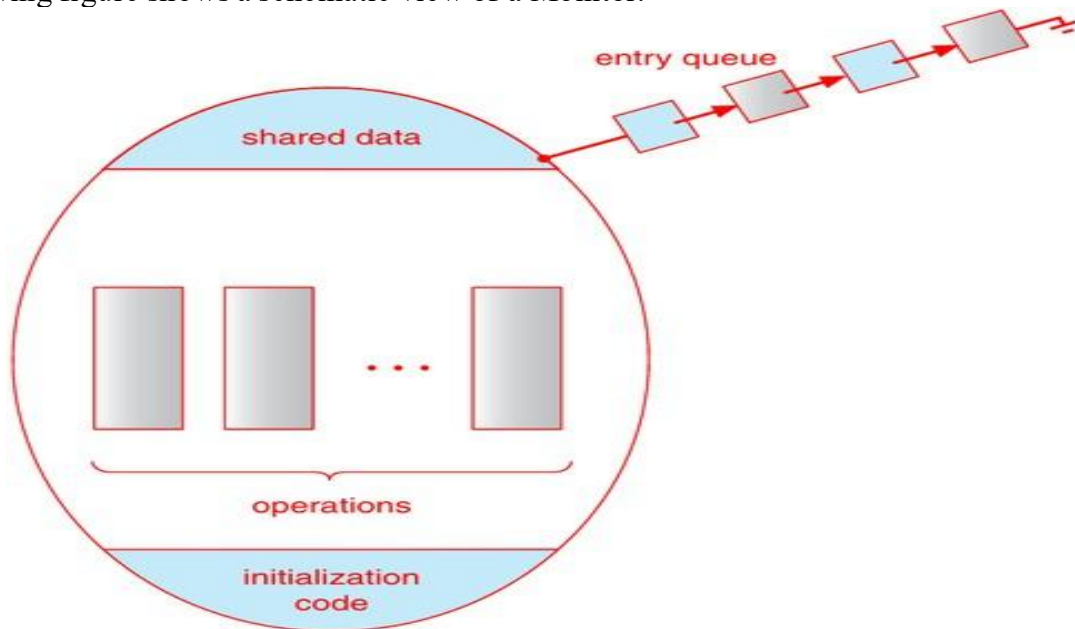
Initialization Code(....)

{

}

}

The following figure shows a schematic view of a Monitor.



Only one process can be active in a monitor at a time. Other processes that need to access the shared variables in a monitor have to line up in a queue and are only provided access when the previous process releases the shared variables.

Classical problems of Synchronization

The following problems of synchronization are considered as classical problems. These problems are used

for testing nearly every newly proposed synchronization scheme.

1. The Bounded-buffer(Producer-Consumer) Problem

2. The Readers-Writers Problem

3. The Dining-Philosophers Problem

1. The Bounded-buffer(Producer-Consumer) Problem

Bounded Buffer problem is also called producer consumer problem. The producer consumer problem is a synchronization problem. There is a fixed size buffer and the producer produces items and enters them into the buffer. The consumer removes the items from the buffer and consumes them.

A producer should not produce items into the buffer when the consumer is consuming an item from the buffer and vice versa. So the buffer should only be accessed by the producer or consumer at a time.

The producer consumer problem can be resolved using semaphores. The codes for the producer and consumer process are given as follows:

Producer Process :-

The code that defines the producer process is given below:

```
do {  
  
    . PRODUCE ITEM  
  
    .  
  
    wait(empty);  
  
    wait(mutex);  
  
    .  
  
    . PUT ITEM IN BUFFER  
  
    .  
  
    signal(mutex);  
  
    signal(full);  
  
} while(1);
```

In the above code, mutex, empty and full are semaphores. Here mutex is initialized to 1, empty is initialized to n (maximum size of the buffer) and full is initialized to 0.

The mutex semaphore ensures mutual exclusion. The empty and full semaphores count the number of empty and full spaces in the buffer.

After the item is produced, wait operation is carried out on empty. This indicates that the empty space in the buffer has decreased by 1. Then wait operation is carried out on mutex so that consumer process cannot interfere.

After the item is put in the buffer, signal operation is carried out on mutex and full. The former indicates that consumer process can now act and the latter shows that the buffer is full by 1.

Consumer Process:-

The code that defines the consumer process is given below:

```
do {  
  
    wait(full);  
  
    wait(mutex);  
  
    . .  
  
    . REMOVE ITEM FROM BUFFER  
  
    .  
  
    signal(mutex);  
    signal(empty);  
  
    .  
  
    . CONSUME ITEM  
  
    .  
  
} while(1);
```

The wait operation is carried out on full. This indicates that items in the buffer have decreased by 1. Then wait operation is carried out on mutex so that producer process cannot interfere.

Then the item is removed from buffer. After that, signal operation is carried out on mutex and empty. The former indicates that consumer process can now act and the latter shows that the empty space in the buffer has increased by 1.

2.The Readers-Writers Problem

Suppose that a database is to be shared among several concurrent processes. Some of these processes may want only to read the database, whereas others may want to update (that is, to read and write) the database. We distinguish between these two types of processes by referring to the former as readers and to the latter as writers. Precisely in OS we call this situation as the readers-writers problem. Problem parameters:

- One set of data is shared among a number of processes.
- Once a writer is ready, it performs its write. Only one writer may write at a time.
- If a process is writing, no other process can read it.
- If at least one reader is reading, no other process can write.
- Readers may not write and only read.

The readers-writers problem relates to an object such as a file that is shared between multiple processes. Some of these processes are readers i.e. they only want to read the data from the object and some of the processes are writers i.e. they want to write into the object.

The readers-writers problem is used to manage synchronization so that there are no problems with the object data. For example - If two readers access the object at the same time there is no problem. However if two writers or a reader and writer access the object at the same time, there may be problems.

To solve this situation, a writer should get exclusive access to an object i.e. when a writer is accessing the object, no reader or writer may access it. However, multiple readers can access the object at the same time.

This can be implemented using semaphores. The codes for the reader and writer process in the reader-writer problem are given as follows –

Reader Process :-

The code that defines the reader process is given below:

```
wait (mutex);

rc ++;

if (rc == 1)

wait (wrt);

signal(mutex);

.

. READ THE OBJECT

.

wait(mutex);

rc --;

if (rc == 0)

signal (wrt);

signal(mutex);
```

In the above code, mutex and wrt are semaphores that are initialized to 1. Also, rc is a variable that is initialized to 0. The mutex semaphore ensures mutual exclusion and wrt handles the writing mechanism and is common to the reader and writer process code.

The variable rc denotes the number of readers accessing the object. As soon as rc becomes 1, wait operation is used on wrt. This means that a writer cannot access the object anymore. After the read operation is done, rc is decremented. When rc becomes 0, signal operation is used on wrt. So a writer can access the object now.

Writer Process:-

The code that defines the writer process is given below:

```
wait(wrt);

.
```

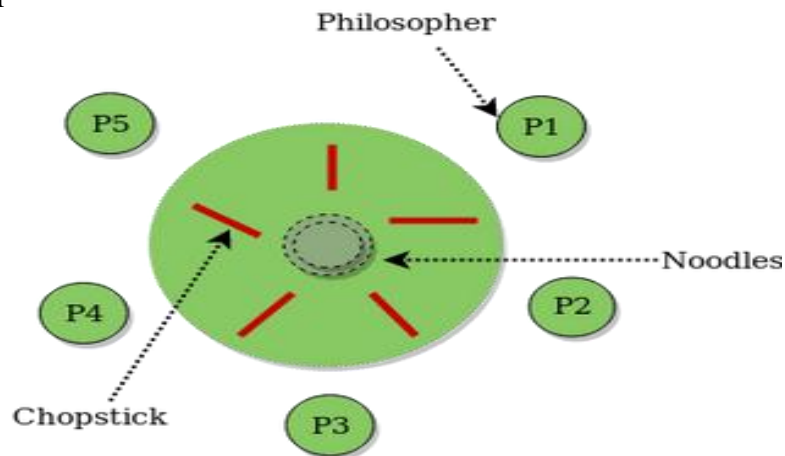
. WRITE INTO THE OBJECT

signal(wrt);

If a writer wants to access the object, wait operation is performed on wrt. After that no other writer can access the object. When a writer is done writing into the object, signal operation is performed on wrt.

3. The Dining-Philosophers Problem

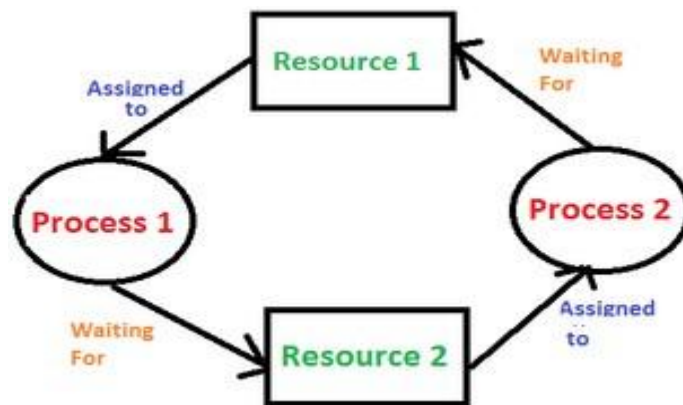
The Dining Philosopher Problem states that 'K' philosophers seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. One chopstick may be picked up by any one of its adjacent followers but not both. This problem involves the allocation of limited resources to a group of processes in a deadlock-free and starvation-free manner.



Several possible remedies to the deadlock problem are the following:

- Allow at most four philosophers to be sitting simultaneously at the table.
- Allow a philosopher to pick up his chopsticks only if both chopsticks are available (to do this, he must pick them up in a critical section).
- Use an asymmetric solution—that is, an odd-numbered philosopher picks up first his left chopstick and then his right chopstick, whereas an even numbered philosopher picks up his right chopstick and then his left chopstick.

Deadlocks



Deadlock :- **Deadlock** is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process. For example, in the below diagram, Process 1 is holding Resource 1 and waiting for resource 2 which is acquired by process 2, and process 2 is waiting for resource 1.

System Model

A system consists of a finite number of resources to be distributed among a number of competing processes. The resources may be partitioned into several types (or classes), each consisting of some number of identical instances. CPU cycles, files, and I/O devices (such as printers and DVD drives) are examples of resource types. If a system has two CPUs, then the resource type CPU has two instances. Similarly, the resource type printer may have five instances.

If a process requests an instance of a resource type, the allocation of any instance of the type should satisfy the request. If it does not, then the instances are not identical, and the resource type classes have not been defined properly. For example, a system may have two printers. These two printers may be defined to be in the same resource class if no one cares which printer prints which output. However, if one printer is on the ninth floor and the other is in the basement, then people on the ninth floor may not see both printers as equivalent, and separate resource classes may need to be defined for each printer.

A process must request a resource before using it and must release the resource after using it. A process may request as many resources as it requires to carry out its designated task. Obviously, the number of resources requested may not exceed the total number of resources available in the system. In other words, a process cannot request three printers if the system has only two. Under the normal mode of operation, a process may utilize a resource in only the following sequence :

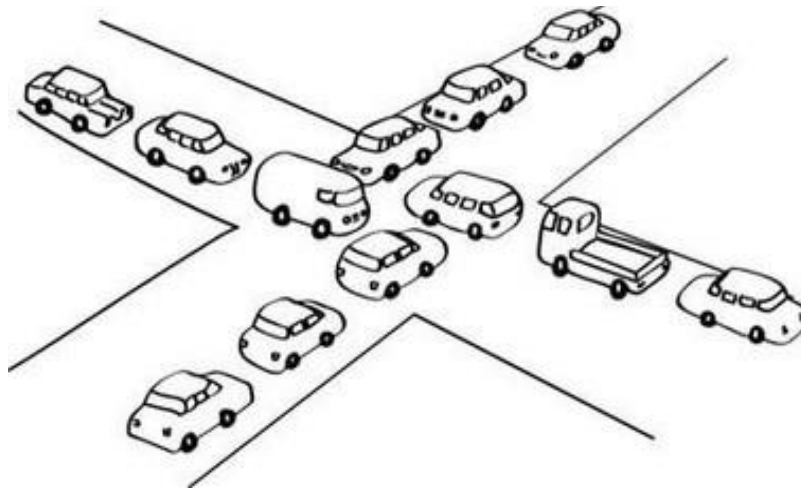
- **Request** - The process requests the resource. If the request cannot be granted immediately (for example, if the resource is being used by another process), then the requesting process must wait until it can acquire the resource.
- **Usage** - The process can operate on the resource (for example, if the resource is a printer, the process can print on the printer).
- **Release** - After usage, processes should release the resources so that others can use it. It is done by using system calls like `free()` for memory, `close()` for files, `release()` for devices.

The operating system maintains a table of records to store the information about resources i.e. whether they are free or allocated to some other process. Another table of allocated resources is maintained which stores processes to which they are allocated.

Deadlock Characterization

Deadlock is a situation /condition or problem. In a deadlock, processes never finish executing and system

resources are tied up, preventing other jobs from starting. A deadlock happens in operating system when two or more processes need some resource to complete their execution that is held by the other process.

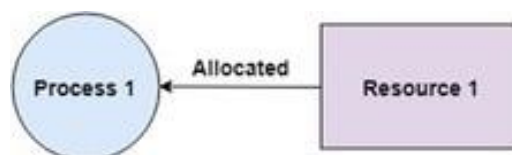


Necessary Conditions for Deadlock

Deadlock can arise if the following four conditions hold simultaneously in the system.

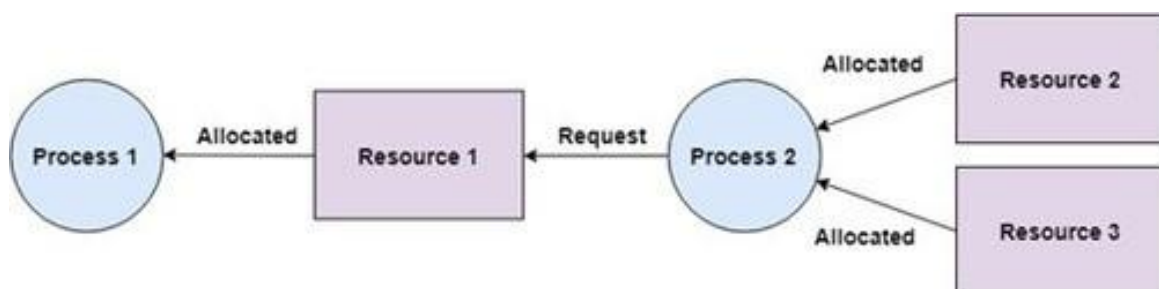
Mutual Exclusion

There should be a resource that can only be held by one process at a time. In the diagram below, there is a single instance of Resource 1 and it is held by Process 1 only.



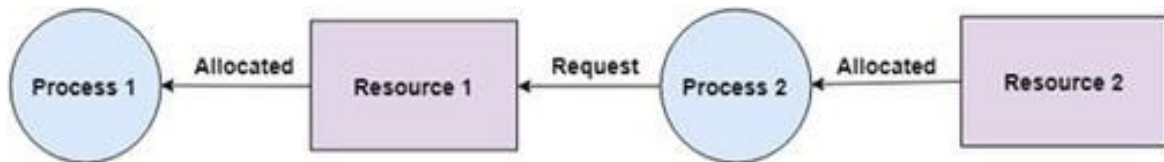
Hold and Wait

A process can hold multiple resources and still request more resources from other processes which are holding them. In the diagram given below, Process 2 holds Resource 2 and Resource 3 and is requesting the Resource 1 which is held by Process 1.



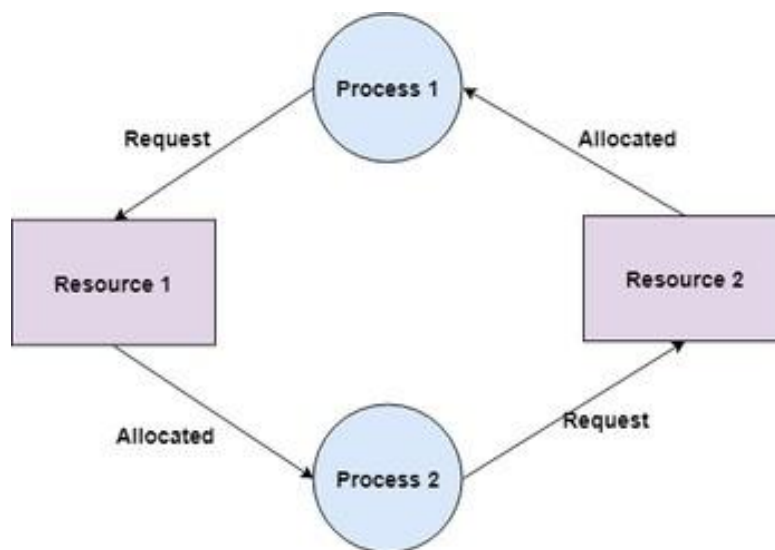
No Preemption

A resource cannot be preempted from a process by force. A process can only release a resource voluntarily. In the diagram below, Process 2 cannot preempt Resource 1 from Process 1. It will only be released when Process 1 relinquishes it voluntarily after its execution is complete.



Circular Wait

A process is waiting for the resource held by the second process, which is waiting for the resource held by the third process and so on, till the last process is waiting for a resource held by the first process. This forms a circular chain. For example: Process 1 is allocated Resource2 and it is requesting Resource 1. Similarly, Process 2 is allocated Resource 1 and it is requesting Resource 2. This forms a circular wait loop.



Methods for handling deadlocks

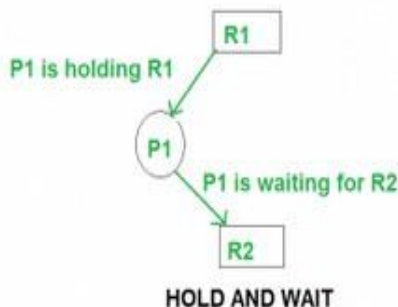
Deadlocks can be handled by using the following three techniques:

1. **Deadlock Prevention**
2. **Deadlock Avoidance**
3. **Deadlock Detection and Recovery**

1. Deadlock Prevention :-

We can prevent Deadlock by eliminating any of the following four conditions.

- **Eliminate Mutual Exclusion** :-It is not possible to dis-satisfy the mutual exclusion because some resources, such as the tape drive and printer, are inherently non-shareable.
- **Eliminate Hold and wait** :-Allocate all required resources to the process before the start of its execution, this way hold and wait condition is eliminated but it will lead to low device utilization. for example, if a process requires printer at a later time and we have allocated printer before the start of its execution printer will remain blocked till it has completed its execution. The process will make a new request for resources after releasing the current set of resources. This solution may lead to starvation.



- **Eliminate No Preemption** :-Preempt resources from the process when resources required by other high priority processes.
- **Eliminate Circular Wait** :- Each resource will be assigned with a numerical number. A process can request the resources increasing/decreasing. order of numbering. For Example, if P1 process is allocated R5 resources, now next time if P1 ask for R4, R3 lesser than R5 such request will not be granted, only request for resources more than R5 will be granted.

2. Deadlock Avoidance :-

Deadlock avoidance can be done with Banker's Algorithm.

Banker's Algorithm :- Banker's Algorithm is resource allocation and deadlock avoidance algorithm which test all the request made by processes for resources, it checks for the safe state, if after granting request system remains in the safe state it allows the request and if there is no safe state it doesn't allow the request made by the process.

Inputs to Banker's Algorithm :-

1. Max need of resources by each process.
2. Currently, allocated resources by each process.
3. Max free available resources in the system.

The request will only be granted under the below condition :-

1. If the request made by the process is less than equal to max need to that process.
2. If the request made by the process is less than equal to the freely available resource in the system.

Example:

Total resources in system:

A B C D

6 5 7 6

Available system resources are:

A B C D

3 1 1 2

Processes (currently allocated resources):

A B C D

P1 1 2 2 1

P2 1 0 3 3

P3 1 2 1 0

Processes (maximum resources):

A B C D

P1 3 3 2 2

P2 1 2 3 4

P3 1 3 5 0

Need = maximum resources - currently allocated resources.

Processes (need resources):

A B C D

P1 2 1 0 1

P2 0 2 0 1

P3 0 1 4 0

The Banker's Algorithm consists of the following two algorithms:

1. Resource-Request Algorithm
2. Safety Algorithm

Resource-Request Algorithm :-

Whenever a process makes a request of the resources then this algorithm checks that if the resource can be allocated or not. It includes the following three steps:

1. The algorithm checks that if the request made is valid or not. A request is valid if the number of requested resources of each resource type is less than the Need (which was declared previously by the process). If it is a valid request then step 2 is executed else aborted.
2. Here, the algorithm checks that if the number of requested instances of each resource is less than the available resources. If not then the process has to wait until sufficient resources are available else go to step 3.
3. Now, the algorithm assumes that the resources have been allocated and modifies the data structure accordingly.

$\text{Available} = \text{Available} - \text{Request}(i)$

$\text{Allocation}(i) = \text{Allocation}(i) + \text{Request}(i)$

$\text{Need}(i) = \text{Need}(i) - \text{Request}(i)$

After the allocation of resources, the new state formed may or may not be a safe state. So, the safety algorithm is applied to check whether the resulting state is a safe state or not.

Safety Algorithm :-

The safety algorithm is applied to check whether a system is in a safe state or not. This algorithm involves the following four steps:

1. Suppose currently all the processes are to be executed. Define two data structures as work and finish as vectors of length m (where m is the length of Available vector) and n (is the number of processes to be executed).

$\text{Work} = \text{Available}$

$\text{Finish}[i] = \text{false}$ for $i = 0, 1, \dots, n-1$.

1. This algorithm will look for a process that has Need value less than or equal to the Work. So, in this step, we will find an index i such that

$\text{Finish}[i] == \text{false} \ \&\& \ \text{Need}[i] \leq \text{Work}$

If no such “i” is present then goto step4 else to step3.

1. The process 'i' selected in the above step runs and finishes its execution. Also, the resources allocated to it get free. The resources which get free are added to the Work and Finish(i) of the process is set as true. The following operations are performed

$Work = Work + Allocation$

$Finish[i] = true$

After performing the 3rd step goto step2.

1. If all the processes are executed in some sequence then it is said to be a safe state. Or, we can say that if $Finish[i] == true$ for all i, Then the system is said to be in a safe state.

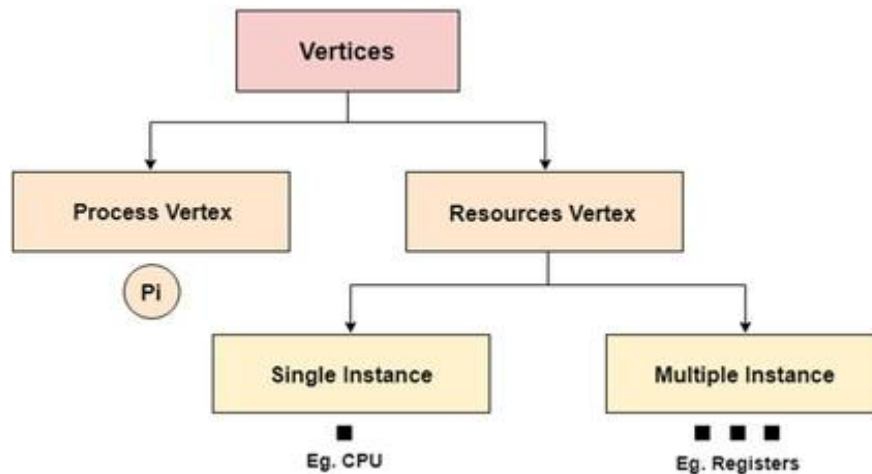
3. Deadlock Detection And Recovery :-

This approach let the processes fall in deadlock and then periodically check whether deadlock occur in the system or not. If it occurs then it applies some of the recovery methods to the system to get rid of deadlock.

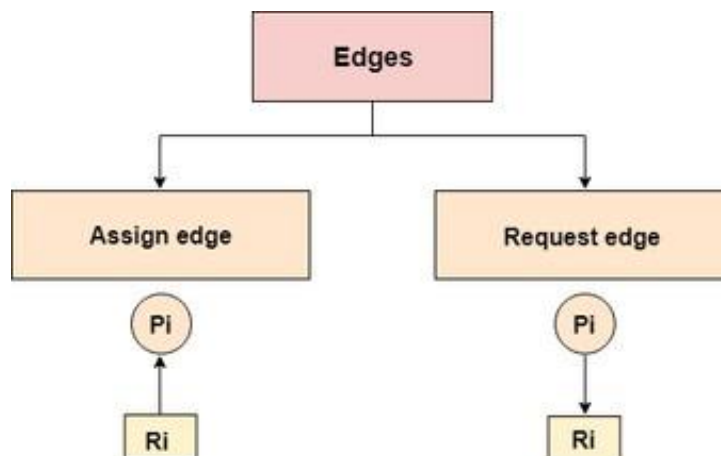
In single instanced resource types, if a cycle is being formed in the system then there will definitely be a deadlock. On the other hand, in multiple instanced resource type, detecting a cycle is not just enough. We have to apply the safety algorithm on the system by converting the resource allocation graph into the allocation matrix and request matrix.

Resource-Allocation Graph (RAG)

The resource allocation graph is the pictorial representation of the state of a system. As its name suggests, the resource allocation graph is the complete information about all the processes which are holding some resources or waiting for some resources. It also contains the information about all the instances of all the resources whether they are available or being used by the processes. In Resource allocation graph, the process is represented by a Circle while the Resource is represented by a rectangle.

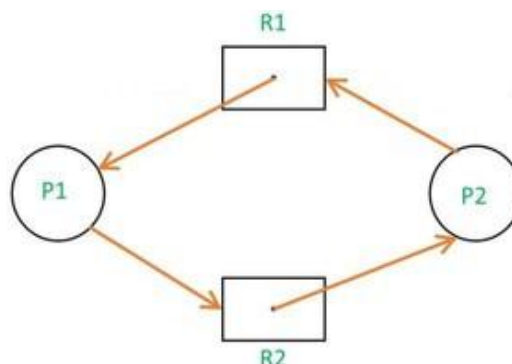


Vertices are mainly of two types, Resource vertex and process vertex. Each of them will be represented by a different shape. Circle represents process while rectangle represents resource. A resource can have more than one instance. Each instance will be represented by a dot inside the rectangle.

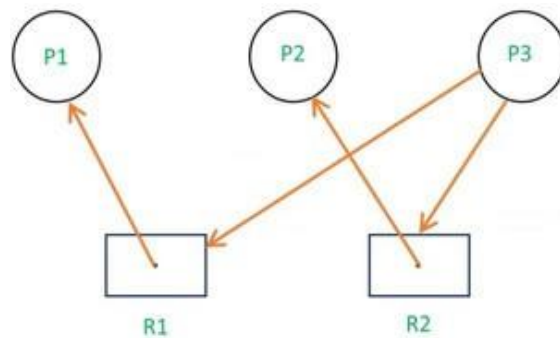


Edges in RAG are also of two types, one represents Assignment Edge and other represents Request Edge for a resource. A resource is shown as assigned to a process if the tail of the arrow is attached to an instance to the resource and the head is attached to a process. A process is shown as waiting for a resource if the tail of an arrow is attached to the process while the head is pointing towards the resource.

Single instances RAG



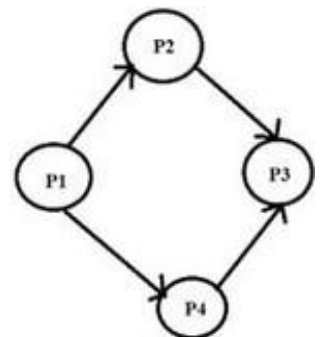
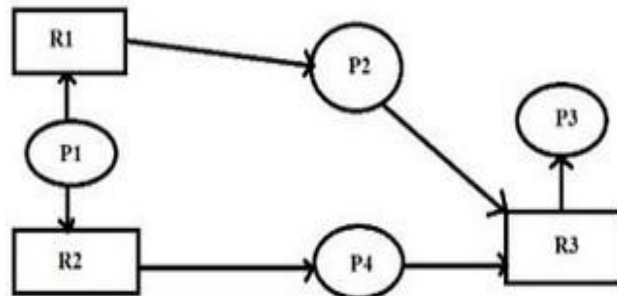
If there is a cycle in the Resource Allocation Graph and each resource in the cycle provides only one instance, then the processes will be in deadlock. For example, if process P1 holds resource R1, process P2 holds resource R2 and process P1 is waiting for R2 and process P2 is waiting for R1, then process P1 and process P2 will be in deadlock.



Here's another example that shows Processes P1 and P2 acquiring resources R1 and R2 while process P3 is waiting to acquire both resources. In this example, there is no deadlock because there is no circular dependency. So cycle in single-instance resource type is the sufficient condition for deadlock.

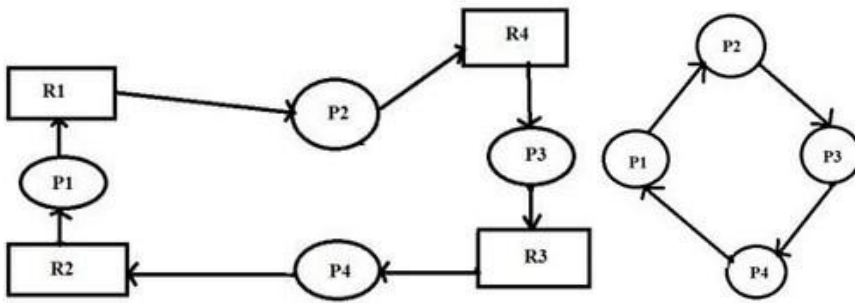
Wait-for Graph

Wait-for graph is an enhanced version of Resource Allocation Graph as it does not includes resources.



Resource Allocation Graph Wait-for Graph

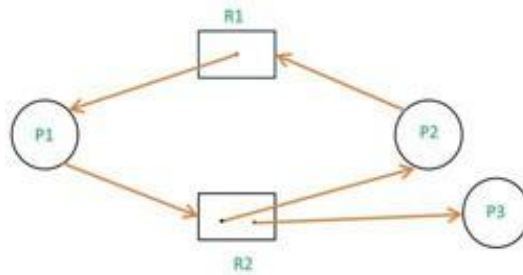
If there is a cycle condition, that if there is a continuous flow of a process in one direction it means cycle condition exists and wait for the graph is in a deadlock condition. In the above graph, shows there is no deadlock state because there is no continuous flow observed in wait for the graph.



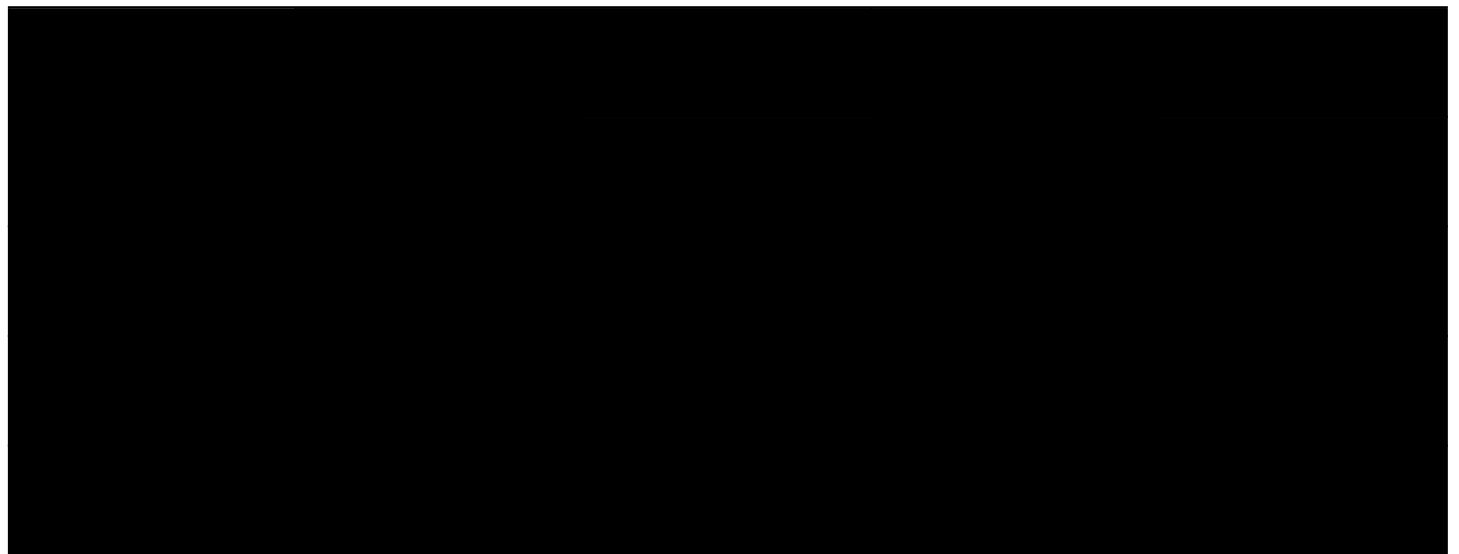
Resource Allocation Graph Wait-for Graph

Deadlock condition has occurred because there is a continuous flow of cycle from P1 to P4.

Multiple-instances RAG



From the above example, it is not possible to say the Resource Allocation Graph is in a safe state or in an unsafe state. So to see the state of this RAG, let's construct the allocation matrix and request matrix.



The total number of processes are three; P1, P2 & P3 and the total number of resources are two; R1 & R2.

Allocation Matrix

- For constructing the allocation matrix, just go to the resources and see to which process it is allocated.
- R1 is allocated to P1, therefore write 1 in allocation matrix and similarly, R2 is allocated to P2 as well as P3 and for the remaining element just write 0.

Request Matrix

In order to find out the request matrix, you have to go to the process and see the outgoing edges. P1 is requesting resource R2, so write 1 in the matrix and similarly, P2 requesting R1 and for the remaining element write 0. So now available resource is=(0, 0).

Available = 0	0	(As P3 does not require any extra resource to complete the execution and after completion P3 release its own resource)
P3	0 1	
New Available = 0	1	(As using new available resource we can satisfy the requirement of process P1 and P1 also release its previous resource)
P1	1 0	
New Available = 1	1	(Now easily we can satisfy the requirement of process P2)
P2	0 1	
New Available = 1	2	

So, there is no deadlock in this RAG. Even though there is a cycle, still there is no deadlock. Therefore in multi-instance resource cycle is not sufficient condition for deadlock.

Recovery from Deadlock

When a detection algorithm determines that a deadlock exists, several alternatives are available. There are two options for breaking a deadlock. One is simply to kill one or more Processes to break the circular wait. The other is to preempt some resources from one or more of the deadlocked processes.

Resource Preemption :-

To eliminate deadlocks using resource preemption, we deallocated or preempt some resources from processes and give these resources to other processes until the deadlock cycle is broken. This method will raise the following three issues:

- **Selection of victim** we need to decide which process or which resources are to be preempted; the decision is based on cost factor which includes the number of resources, a deadlocked process is holding and CPU time consumed by it etc.
- **Rollback**: the process which was preempted cannot continue normal execution, because its resources are taken back. Hence, we need to rollback to some previous check point or total rollback to start it from the beginning.
- **Starvation**: in a system, it may happen that same process is always picked as a victim. As a result, that process will never complete its designated task. This situation is called Starvation and must be avoided. One solution is that a process must be picked as a victim only a finite number of times.

Process Termination :-

To eliminate deadlocks by aborting a process, we use one of two methods. In both methods, the system reclaims all resources allocated to the terminated processes.

Abort all deadlocked processes: terminate all deadlocked processes which will break deadlock immediately, but it is a bit expensive because there may be some processes which have been executing for a long time consuming considerable CPU time and their termination will result in wasting those CPU cycles.

Abort one process at a time until the deadlock cycle is eliminated: in order to overcome the drawback of the above method, this method terminates one process at a time until deadlock is recovered. however it has

some overhead since ,after terminating each process detection algorithm has to be executed for deciding to further terminate the processes or not. This method is slower than the first method.